Enterprise AI Solution Report

Requirement Analysis

Analysis of the Requirement

Project Objective:
The project objective is to build an Employee Management System, which suggests that the primary goal is to create a system that effectively manages employee-related activities within an organization.

Key Features:
The key features of the system are:

1. **Login**: This feature implies that the system will have user authentication and authorization mechanisms to ensure that only authorized personnel can access the system.
2. **Attendance**: This feature suggests that the system will track employee attendance, which can be used for various purposes such as payroll processing, leave management, and performance evaluation.
3. **Leave Management**: This feature implies that the system will manage employee leave requests, approvals, and notifications.
4. **Reports**: This feature suggests that the system will generate various reports based on employee data, attendance, and leave management.

Stakeholders:
The stakeholders of the Employee Management System are likely to be:

1. **HR Department**: Responsible for managing employee data, attendance, and leave management.
2. **Management**: Interested in accessing reports and analytics to make informed decisions.
3. **Employees**: Need to access the system for login, attendance tracking, and leave management.
4. **IT Department**: Responsible for maintaining the system, ensuring data security, and providing technical support.

Functional Requirements:

1. **User Authentication**: The system should have a robust user authentication mechanism to ensure that only authorized personnel can access the system.
2. **Attendance Tracking**: The system should be able to track employee attendance, including clock-in and clock-out times, breaks, and holidays.
3. **Leave Request and Approval**: The system should allow employees to request leave, and managers to approve or reject leave requests.
4. **Leave Management**: The system should be able to manage employee leave balances, including accruals and carryovers.
5. **Reporting**: The system should generate various reports based on employee data, attendance, and leave management.
6. **API Integration**: The system should have a REST API to enable integration with other systems and applications.
7. **Database Management**: The system should have a robust database management system to ensure data security and integrity.

Additional Considerations:

1. **Data Security**: The system should have robust security measures in place to protect

employee data and prevent unauthorized access.
2. **Scalability**: The system should be designed to scale with the organization's growth, ensuring that it can handle increased user traffic and data volume.
3. **User Experience**: The system should have an intuitive user interface and user experience to ensure that employees and managers can easily navigate and use the system.

User Stories

Here are 10 user stories for the Employee Management System:

User Story 1:
As an HR Manager,
I want to be able to login to the system with a unique username and password,
So that I can securely access and manage employee data.

User Story 2:
As an HR Manager,
I want to be able to view and manage employee attendance records,
So that I can track employee work hours and attendance patterns.

User Story 3:
As an HR Manager,
I want to be able to create, manage and assign leave for employees,
So that I can ensure compliance with company leave policies and employee entitlements.

User Story 4:
As an HR Manager,
I want to be able to generate reports on employee attendance and leave,
So that I can analyze trends and make informed decisions.

User Story 5:
As a System Administrator,
I want to be able to configure user roles and permissions,
So that I can control access to sensitive employee data and system features.

User Story 6:
As an Employee,
I want to be able to view my attendance history,
So that I can track my work hours and attendance patterns.

User Story 7:
As an Employee,
I want to be able to request leave online,
So that I can submit leave requests efficiently and conveniently.

User Story 8:
As an HR Manager,
I want to be able to receive notifications when an employee submits a leave request,
So that I can review and approve or reject leave requests in a timely manner.

User Story 9:
As an HR Manager,
I want to be able to export reports to CSV or Excel,
So that I can analyze data and share insights with stakeholders.

User Story 10:
As a System Administrator,
I want to be able to integrate the Employee Management System with existing HR systems,
So that I can streamline employee data management and reduce data duplication.

API Design

Employee Management System REST APIs

Database Design

We will use a MySQL database for our Employee Management System. The database schema includes the following tables:

```sql
CREATE TABLE users (
id INT PRIMARY KEY AUTO_INCREMENT,
username VARCHAR(255) NOT NULL,
password VARCHAR(255) NOT NULL,
role ENUM('admin', 'user') NOT NULL
);

CREATE TABLE employees (
id INT PRIMARY KEY AUTO_INCREMENT,
name VARCHAR(255) NOT NULL,
email VARCHAR(255) NOT NULL,
department VARCHAR(255) NOT NULL,
designation VARCHAR(255) NOT NULL
);

CREATE TABLE attendance (
id INT PRIMARY KEY AUTO_INCREMENT,
employee_id INT NOT NULL,
date DATE NOT NULL,
present ENUM('yes', 'no') NOT NULL,
FOREIGN KEY (employee_id) REFERENCES employees(id)
);

CREATE TABLE leave (
id INT PRIMARY KEY AUTO_INCREMENT,
employee_id INT NOT NULL,
date DATE NOT NULL,
reason VARCHAR(255) NOT NULL,
status ENUM('approved', 'rejected') NOT NULL,
FOREIGN KEY (employee_id) REFERENCES employees(id)
);

CREATE TABLE reports (
id INT PRIMARY KEY AUTO_INCREMENT,
employee_id INT NOT NULL,
month INT NOT NULL,
year INT NOT NULL,
attendance INT NOT NULL,
FOREIGN KEY (employee_id) REFERENCES employees(id)
);
```

Login Endpoint

Endpoint: /login
Method: POST

Request Body:

```json
{
"username": "string",
"password": "string"
}
```

Response:

```json
{
"token": "string",
"role": "admin/user"
}
```

Login API

```java
@PostMapping("/login")
public ResponseEntity> login(@RequestBody LoginRequest request) {
// Authenticate user
User user = authenticationService.authenticate(request.getUsername(), request.getPassword());
if (user == null) {
return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
}
// Generate token
String token = jwtTokenGenerator.generateToken(user);
// Return token and role
return ResponseEntity.ok(Map.of("token", token, "role", user.getRole()));
}

public class LoginRequest {
private String username;
private String password;
// Getters and setters
}
```

Attendance Endpoint

Endpoint: /attendance
Method: POST
Request Body:

```json
{
"employee_id": 1,
"date": "2022-01-01",
"present": "yes/no"
}
```

Response:

```json
{
"message": "Attendance updated successfully"
}
```

Attendance API

```java
@PostMapping("/attendance")
public ResponseEntity updateAttendance(@RequestBody AttendanceRequest request) {
// Validate request
if (request.getPresent() == null || request.getDate() == null) {
return ResponseEntity.badRequest().build();
}
// Update attendance
attendanceService.updateAttendance(request.getEmployeeId(), request.getDate(),
request.getPresent());
return ResponseEntity.ok("Attendance updated successfully");
}

public class AttendanceRequest {
private Long employeeId;
private Date date;
private String present;
// Getters and setters
}
```

Leave Endpoint

Endpoint: /leave
Method: POST
Request Body:

```json
{
"employee_id": 1,
"date": "2022-01-01",
"reason": "string",
"status": "approved/rejected"
}
```

Response:

```json
{
"message": "Leave updated successfully"
}
```

Leave API

```java
```

```
@PostMapping("/leave")
public ResponseEntity updateLeave(@RequestBody LeaveRequest request) {
// Validate request
if (request.getDate() == null || request.getReason() == null) {
return ResponseEntity.badRequest().build();
}
// Update leave
leaveService.updateLeave(request.getEmployeeId(), request.getDate(), request.getReason(),
request.getStatus());
return ResponseEntity.ok("Leave updated successfully");
}

public class LeaveRequest {
private Long employeeId;
private Date date;
private String reason;
private String status;
// Getters and setters
}
```

Reports Endpoint

Endpoint: /reports
Method: GET
Request Parameters:

```json
{
"employee_id": 1,
"month": 1,
"year": 2022
}
```

Response:

```json
{
"attendance": 10,
"leave": 5
}
```

Reports API

```java
@GetMapping("/reports")
public ResponseEntity> getReports(@RequestParam Long employeeId, @RequestParam Integer
month, @RequestParam Integer year) {
// Validate request
if (employeeId == null || month == null || year == null) {
return ResponseEntity.badRequest().build();
}
// Get reports
Map reports = reportService.getReports(employeeId, month, year);
```

```
return ResponseEntity.ok(reports);
}
```

Reports Service

```java
@Service
public class ReportService {
// Get reports for an employee
public Map getReports(Long employeeId, Integer month, Integer year) {
// Query database
Map reports = new HashMap<>();
reports.put("attendance", attendanceService.getAttendance(employeeId, month, year));
reports.put("leave", leaveService.getLeave(employeeId, month, year));
return reports;
}
}
```

Attendance Service

```java
@Service
public class AttendanceService {
// Get attendance for an employee
public int getAttendance(Long employeeId, Integer month, Integer year) {
// Query database
return attendanceRepository.countByEmployeeIdAndDate(employeeId, month, year);
}
}
```

Leave Service

```java
@Service
public class LeaveService {
// Get leave for an employee
public int getLeave(Long employeeId, Integer month, Integer year) {
// Query database
return leaveRepository.countByEmployeeIdAndDate(employeeId, month, year);
}
}
```

Database Connection

```java
@Configuration
public class DatabaseConfig {
@Bean
public DataSource dataSource() {
// Configure database connection
return DataSourceBuilder.create()
.driverClassName("com.mysql.cj.jdbc.Driver")
```

```
.url("jdbc:mysql://localhost:3306/employee_management_system")
.username("username")
.password("password")
.build();
}
}
```

Swagger API Documentation

```java
@Configuration
@EnableSwagger2
public class SwaggerConfig {
@Bean
public Docket api() {
return new Docket(DocumentationType.SWAGGER_2)
.select()
.apis(RequestHandlerSelectors.any())
.paths(PathSelectors.any())
.build();
}
}
```

Application Configuration

```java
@SpringBootApplication
public class EmployeeManagementSystemApplication {
public static void main(String[] args) {
SpringApplication.run(EmployeeManagementSystemApplication.class, args);
}
}
```

Dependencies

```xml


org.springframework.boot
spring-boot-starter-web


org.springframework.boot
spring-boot-starter-data-jpa


mysql
mysql-connector-java
runtime


io.springfox
springfox-boot-starter
```

io.jsonwebtoken
jjwt

```
```

Architecture Diagram

flowchart TD
A[Requirement PDF] --> B[PDF Reader]
B --> C[Manager Agent]
C --> D[Requirement Agent]
C --> E[User Story Agent]
C --> F[API Design Agent]
D --> G[Report Generator]
E --> G
F --> G
G --> H[Enterprise PDF Report]